Lab 4

Implementation of the Concept of Classes in Java

Objective:

To understand the fundamental concepts of **classes and objects** in Java and implement them through practical coding exercises

1. Introduction to Classes and Objects in Java

What is a Class?

A **class** is a **blueprint** for creating objects. It defines properties (variables) and methods that an object can have.

Types of Classes in OOP

A. **Regular Class**

1. A normal class that contains variables and methods.
2. Objects can be created from it.

B. **Abstract Class**

1. A class that cannot be used to create objects directly.
2. It may have abstract methods (methods without implementation).
3. Other classes must extend it and provide method definitions.

C. **Interface**

1. A blueprint that contains only method declarations (without implementation).
2. Other classes must implement it and define the methods.
3. Used to achieve multiple inheritance in Java.

D. **Final Class**

1. A class that cannot be inherited by any other class.
2. Used when we want to prevent further modifications.

E. **Static Class**

1. A class that cannot be instantiated (no objects can be created).
2. It contains only static methods and variables.

F. **Inner Class (Nested Class)**

1. A class that is defined inside another class.
2. Used to group related classes together.

G. **Anonymous Class**

1. A class without a name, created for one-time use.
2. Usually used with interfaces or abstract classes for quick implementation.

➢ What is an Object?

An **object** is an instance of a class. When we create an object, we allocate memory for the properties defined in the class.

2. Understanding the Key Concepts

➢ Class Definition in Java:

A class is created using the **class** keyword, followed by a name. The class name should start with an uppercase letter (as per Java naming conventions).

Syntax:

```
class ClassName {
    // Variables (Attributes)
    // Methods (Functions)
}
```

➢ Creating an Object in Java

Objects are created using the **new** keyword, which allocates memory for the object.

Syntax:

```
ClassName objectName = new ClassName();
```

- **ClassName** → The name of the class.
- **objectName** → The name of the object (any valid variable name).
- **new** → The keyword that creates a new object.
- **ClassName()** → This is the constructor that initializes the object.

➢ Methods in Java

◆ What is a Method?

A method in Java is a block of code that performs a specific task. It helps in **code reusability**, meaning we can write code once and use it multiple times.

◆ How to Create a Method in Java?

A method is created inside a **class** using the following

syntax:

```
returnType methodName(parameters) {
    // Method body (code to execute)
}
```

◆ Parts of a Method

1. **Return Type** → Specifies the type of value the method will return (**e.g., int, String, void** if no value is returned).
2. **Method Name** → The unique name of the method.
3. **Parameters (Optional)** → Inputs that the method takes to process (inside parentheses).
4. **Method Body** → The actual code that executes when the method is called.

➢ Types of Methods:

Predefined (Built-in)
User-Defined
Parameterized
Methods with Return Type
Void Methods
Static Methods
Instance Methods
Constructors

1. Predefined Methods (Built-in Methods)

These are methods already defined in Java, like print(), length(), sqrt(), etc.

Example:

```
System.out.println("Hello, World!");
// println() is a built-in method
```

2. User-Defined Methods

These are methods created by the programmer to perform specific tasks.

Syntax:

```
returnType methodName() {
    // Code to execute
    return value;  // If return type is not void
}
```

Example:

```
int addNumbers() {
    return 5 + 10;  // Returns 15
}
```

3. Parameterized Methods

These methods take parameters (inputs) to perform operations.

Syntax:

```
returnType methodName(dataType parameter1, dataType parameter2) {
    // Code to execute
}
```

Example:

```
void greet(String name) {
    System.out.println("Hello, " + name);
}
```

4. Method with Return Type

These methods return a value after execution.

Syntax:

```
returnType methodName() {
    return value;
}
```

Example:

```
int square(int num) {
    return num * num;  // Returns the square of the number
}
```

5. Void Methods (No Return Value)

These methods **do not return** any value. They are used to perform actions.

Syntax:

```
void methodName() {
    // Code to execute
}
```

Example:

```
void displayMessage() {
    System.out.println("This is a void method!");
}
```

6. Static Methods

These methods **belong to the class**, not to an object, and can be called without creating an object.

Syntax:

```
static returnType methodName() {
    // Code to execute
}
```

Example:

```
static void showMessage() {
    System.out.println("This is a static method.");
}
```

7. Instance Methods

These methods **belong to an object** and can be accessed only by creating an object of the class.

Syntax:

```
returnType methodName() {
    // Code to execute
}
```

Example:

```
class Example {
    void instanceMethod() {
        System.out.println("This is an instance method.");
    }
}
```

8. Constructor (Special Method)

A constructor is a **special method** used to initialize an object.

Syntax:

```
ClassName() {
    // Constructor body
}
```

Example:

```
class Car {
    Car() {
        System.out.println("Car object created!");
    }
}
```

3. Example: Creating a Simple Class and Object

Let's create a **Student** class with attributes (name and age) and a method (**displayInfo()**).

```
class Student {
    String name;  // Variable name ha jo store kra ga student's name
    int age;      // Variable name ha jo store kra ga student's age

    // Method bania ha to display student information
    void displayInfo() {
        System.out.println("Student Name: " + name);
        System.out.println("Student Age: " + age);
    }
}
// Main class to execute the program
public class StudentTest {
    public static void main(String[] args) {
        // Creating an object of Student class
        Student student1 = new Student();

        // Assigning values to object properties
        student1.name = "Ali";
        student1.age = 20;
        // Calling the method to display student info
        student1.displayInfo();
    }
}
```

Explanation of Code:

1. We define a **class** named **Student**.
2. Inside the class, we declare **variables** name and age.
3. We define a method **displayInfo()** to print student details.
4. In the **main()** method:
    a. We create an object of **Student** using **new Student()**;
    b. We assign values to the object's attributes.
    c. We call the method **displayInfo()** to print details.

4. Methods most commonly used in Java

A **method** is a function inside a class that performs some actions.

Types of Methods:

1. **Instance Methods** - Need an object to be called.
2. **Static Methods** - Belong to the class, called using the class name.

Example of a Static Method:

```
class MathOperations {
    static int add(int a, int b) {
        return a + b;
    }
}

public class Test {
    public static void main(String[] args) {
        int sum = MathOperations.add(10, 20);
        System.out.println("Sum: " + sum);
    }
}
```

Key Points:

> **Static methods** belong to the class and are called using **ClassName.methodName()**.
> **Instance methods** require an object to be called

5. Access Modifiers in Java

Access modifiers **control the visibility** of class members.

Modifier	Access Within Class	Access Within Package	Access in Subclass	Access Everywhere
private				
default				
protected				
public				

➢ Getters and Setters in Java

When we **declare a variable as private**, it **cannot be accessed directly** from outside the class.
To **safely read or update** the value, we use:

Getter (get**)** → A method that **returns** the value of a private variable.
Setter (set**)** → A method that **sets** (updates) the value of a private variable

Basic Syntax of Getter and Setter:

```
class Example {
    private int number;  // Private variable

    // Setter method to assign value
    public void setNumber(int num) {
        number = num;
    }
    // Getter method to return value
    public int getNumber() {
        return number;
    }
}
```

Class Declaration (Example**)** → Defines a class as a blueprint.

Private Variable (number**)** → Stores data securely, restricting direct access.

Setter Method (setNumber**)** → Assigns a value to number, ensuring controlled modification.

Getter Method (getNumber**)** → Retrieves the value of number safely.

Example of Access Modifiers

```
class Person {
    private String name; // Private variable

    // Public method to access private variable
    public void setName(String n) {
        name = n;   }
    public void getName() {
        System.out.println("Name: " + name);
    } }
public class TestPerson {
    public static void main(String[] args) {
```

```
        Person p = new Person();
        p.setName("Ahmed");
        p.getName();
    }
}
```

Key Points:

private → Only accessible inside the same class.
public → Accessible from anywhere.
Encapsulation → We use get and set methods to control access to private variables.